5-line beginner summary

1. **EDA means understanding your data before building a model.**
2. **Preprocessing means cleaning and preparing data so ML algorithms can learn properly.**
3. **Feature engineering means creating better input columns to improve model performance.**
4. **Data leakage is a serious mistake where the model accidentally sees future/test information.**
5. **Clean, well-prepared data usually improves accuracy more than changing algorithms.**

---

1. What EDA is

EDA = Exploratory Data Analysis

It means exploring and understanding the dataset before applying Machine Learning.

In simple words:

> EDA is like checking raw material before building a product.

Before training a model, we should know:

- How many rows and columns are there?
- What each column means?
- Which columns are numeric?
- Which columns are categorical?
- Are there missing values?
- Are there duplicate rows?
- Are there outliers?
- Is the target column balanced or imbalanced?
- Are some columns strongly related to the target?

Example:

Suppose we have customer loan data:

Age	Salary	City	Loan Amount	Default
25	40000	Pune	200000	No
45	90000	Mumbai	500000	Yes
32	null	Delhi	300000	No

EDA helps us notice:

- Salary has missing values.
- City is categorical.
- Default is the target column.

- Loan Amount may have outliers.
- Salary and Loan Amount may be important features.

---

2. Why EDA is important before ML

Machine Learning models do not understand messy data well.

If data has missing values, wrong data types, duplicates, or outliers, the model can learn wrong patterns.

Example:

If salary has missing values:

```
Salary = null
```

Some ML models may fail directly.

If age has wrong value:

```
Age = 500
```

The model may assume extremely old customers exist.

If duplicate rows exist:

```
Same customer repeated 10 times
```

The model may give too much importance to that customer type.

EDA helps us find these problems early.

---

3. Understanding data shape, columns and data types

Data shape

Shape tells us:

```
number of rows × number of columns
```

Example:

```
df.shape
```

Output:

```
(10000, 15)
```

Meaning:

```
10000 rows
15 columns
```

Rows are records.

Columns are features.

Columns

Columns tell us what information we have.

Example:

```
df.columns
```

Output:

```
['customer_id', 'age', 'salary', 'city', 'loan_amount', 'default']
```

Here:

- `customer_id` is an identifier
- `age` is numerical
- `salary` is numerical
- `city` is categorical
- `loan_amount` is numerical
- `default` is target

Data types

Data type tells us what kind of values are stored in each column.

Example:

```
df.dtypes
```

Output:

```
age             int64
salary          float64
city            object
loan_amount     float64
default         object
```

Common data types:

Data type	Meaning	Example
int	Whole number	25
float	Decimal number	45.6
object/string	Text value	Mumbai
datetime	Date/time	2026-07-05
boolean	True/False	True

Why this matters:

ML models usually need numbers. Text columns must often be encoded before training.

---

4. Missing values

Missing values mean some data is not available.

Example:

Age	Salary	City
25	40000	Pune
32	null	Delhi
null	70000	Mumbai

Missing values can happen because:

- User did not provide information
- Data collection failed
- Field was not applicable
- Data integration issue happened

Check missing values:

```
df.isnull().sum()
```

Example output:

```
age       10
salary    50
city       5
```

How to handle missing values

Option 1: Remove rows

Useful when missing records are very few.

```
df = df.dropna()
```

But be careful. If many rows are removed, we lose useful data.

Option 2: Fill numerical values

Common methods:

```
df['salary'] = df['salary'].fillna(df['salary'].mean())
```

or

```
df['salary'] = df['salary'].fillna(df['salary'].median())
```

Mean is affected by outliers.

Median is safer when outliers exist.

Option 3: Fill categorical values

Use mode or a new category.

```
df['city'] = df['city'].fillna(df['city'].mode()[0])
```

or

```
df['city'] = df['city'].fillna('Unknown')
```

---

5. Duplicate records

Duplicate records mean the same row appears more than once.

Example:

Customer ID	Age	Salary
C101	25	40000
C101	25	40000

Check duplicates:

```
df.duplicated().sum()
```

Remove duplicates:

```
df = df.drop_duplicates()
```

Why duplicates are dangerous:

If the same record appears many times, the model may learn that pattern too strongly.

Example:

If one customer default record is repeated 50 times, the model may become biased toward default prediction.

---

6. Outliers

Outliers are values that are unusually high or low.

Example:

Age
25
30
35

	Age
	500

Here `500` is an outlier.

Outliers can happen due to:

- Data entry mistakes
- Real rare cases
- Fraud behavior
- System errors

Outliers are not always bad. In fraud detection, outliers may be very important.

How to detect outliers

Method 1: Simple business rule

```
Age should be between 0 and 100
```

If age is 500, it is invalid.

Method 2: Boxplot

A boxplot helps visually identify extreme values.

Method 3: IQR method

IQR = Interquartile Range

```
IQR = Q3 - Q1
```

Values below:

```
Q1 - 1.5 × IQR
```

or above:

```
Q3 + 1.5 × IQR
```

may be outliers.

How to handle outliers

Method	Meaning	When to use
Remove	Delete outlier records	When values are clearly wrong
Cap	Limit extreme values	When values are valid but too extreme

Method	Meaning	When to use
Transform	Apply log/sqrt transformation	When distribution is highly skewed
Keep	Do nothing	When outliers are meaningful

Example:

Salary values:

```
30000, 40000, 50000, 10000000
```

If `10000000` is valid CEO salary, maybe keep it.

If it is a data entry mistake, fix or remove it.

---

7. Categorical variables

Categorical variables are text or group-based values.

Examples:

```
City = Pune, Mumbai, Delhi
Gender = Male, Female
Department = HR, Finance, IT
Product Type = Basic, Premium
```

ML models cannot directly understand text categories.

So we convert them into numbers using encoding.

Types of categorical variables

Nominal categorical variable

No natural order.

Example:

```
City = Pune, Mumbai, Delhi
```

Pune is not greater than Mumbai.

Ordinal categorical variable

Has natural order.

Example:

```
Education = High School < Graduate < Postgraduate
```

Here order matters.

---

8. Numerical variables

Numerical variables contain numbers.

Examples:

```
Age
Salary
Loan Amount
Transaction Amount
Experience
Temperature
```

Numerical variables can be:

Discrete

Countable values.

Example:

```
Number of children = 0, 1, 2, 3
```

Continuous

Can have decimal values.

Example:

```
Salary = 45678.90
Temperature = 32.5
```

For numerical variables, we usually check:

- Minimum value
- Maximum value
- Mean
- Median
- Standard deviation
- Distribution
- Outliers
- Relationship with target

Example:

```
df.describe()
```

---

9. Encoding

Encoding means converting categorical values into numbers.

ML models usually need numerical input.

Label Encoding

Assigns one number to each category.

Example:

City	Encoded City
Pune	0
Mumbai	1
Delhi	2

Problem:

The model may think:

```
Delhi > Mumbai > Pune
```

But city has no order.

So label encoding is not always suitable for nominal categories.

One-Hot Encoding

Creates separate columns for each category.

Example:

City	City_Pune	City_Mumbai	City_Delhi
Pune	1	0	0
Mumbai	0	1	0
Delhi	0	0	1

This is safer for nominal categories.

Example code:

```
pd.get_dummies(df, columns=['city'])
```

Ordinal Encoding

Used when category order matters.

Example:

Education	Encoded
High School	1
Graduate	2
Postgraduate	3

This is okay because education level has order.

---

10. Scaling

Scaling means bringing numerical columns to a similar range.

Example:

Age	Salary
25	40000
40	90000

Salary values are much larger than age values.

Some algorithms may give more importance to salary only because the numbers are bigger.

Scaling helps avoid this issue.

Standardization

Converts values so they have:

```
mean = 0
standard deviation = 1
```

Useful for:

- Logistic Regression
- Linear Regression
- SVM
- KNN
- Neural Networks

Example:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Normalization

Converts values to a fixed range, usually 0 to 1.

Formula:

```
new_value = (value - min) / (max - min)
```

Useful for:

- Neural Networks

- Distance-based algorithms
- Some recommendation systems

Example:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)
```

Algorithms that usually need scaling

Algorithm	Needs scaling?	Why
Linear Regression	Yes	Coefficients become stable
Logistic Regression	Yes	Optimization improves
KNN	Yes	Uses distance
SVM	Yes	Uses distance/margins
Neural Networks	Yes	Training becomes smoother
Decision Tree	Usually no	Splits based on thresholds
Random Forest	Usually no	Tree-based
XGBoost	Usually no	Tree-based

---

11. Feature engineering

Feature engineering means creating useful new columns from existing columns.

It helps the model learn better patterns.

Example:

Original columns:

Date of Birth	Purchase Date
1995-01-01	2026-07-05

New feature:

```
Age = Purchase Date - Date of Birth
```

This is more useful for ML than raw date of birth.

Examples of feature engineering

Example 1: Create age from date of birth

```
df['age'] = current_year - df['birth_year']
```

Example 2: Create income per family member

```
df['income_per_member'] = df['income'] / df['family_size']
```

Example 3: Extract date features

From transaction date:

```
2026-07-05 14:30:00
```

Create:

```
day = 5
month = 7
day_of_week = Sunday
hour = 14
is_weekend = True
```

Example 4: Create text length feature

For customer reviews:

```
df['review_length'] = df['review_text'].str.len()
```

Longer reviews may indicate strong positive or negative opinions.

Example 5: Create ratio features

For loan data:

```
df['loan_to_income_ratio'] = df['loan_amount'] / df['salary']
```

This may be more useful than loan amount alone.

Example 6: Binning

Convert continuous values into groups.

Example:

Age	Age Group
22	Young
45	Middle
70	Senior

Useful when exact value is less important than range.

---

12. Data leakage

Data leakage happens when the model gets information during training that would not be available in real life at prediction time.

This is one of the biggest ML mistakes.

The model may show very high accuracy during testing, but fail badly in production.

Simple example of leakage

Suppose we are predicting loan default.

Target:

```
default = Yes/No
```

Bad feature:

```
recovery_amount_after_default
```

This value is available only after default happens.

If we use it, the model is cheating.

Another example

Predicting whether a customer will leave a company.

Target:

```
churn = Yes/No
```

Bad feature:

```
account_closure_date
```

This directly tells the answer.

Common leakage sources

Leakage source	**Example**
Future information	Using post-event data
Target-derived feature	Using column calculated from target
Preprocessing before split	Scaling full dataset before train/test split
Duplicate leakage	Same customer appears in both train and test
Time leakage	Random split in time-series data
Aggregation leakage	Calculating average using full dataset

Correct principle

> Anything learned from data should be learned only from training data, then applied to test data.

For example, scaler should be fitted only on training data:

```
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Wrong:

```
scaler.fit_transform(full_dataset)
```

This leaks test data information into training.

---

13. Train/test split after preprocessing

This point needs careful understanding.

Some preprocessing steps can happen before train/test split.

Some must happen after split.

Safe before split

Usually safe:

- Removing duplicate exact rows carefully
- Fixing column names
- Correcting data types
- Removing obviously invalid records
- Basic business-rule cleaning
- Dropping irrelevant ID columns

Example:

```
df.columns = df.columns.str.lower()
df['date'] = pd.to_datetime(df['date'])
```

Should happen after split

Must be fitted only on training data:

- Missing value imputation
- Scaling
- Encoding
- Feature selection
- Outlier capping based on statistics
- PCA

- Any transformation that learns from data

Correct process:

```
1. Split data into train and test
2. Fit preprocessing only on train
3. Transform train
4. Transform test using same fitted transformer
5. Train model
6. Evaluate on test
```

Why?

Suppose you fill missing salary using average salary.

Wrong:

```
Average salary calculated using train + test data
```

This allows training process to indirectly see test data.

Correct:

```
Average salary calculated using train data only
```

Then apply that average to both train and test.

---

14. How clean data improves model performance

Clean data helps the model learn real patterns instead of noise.

Dirty data

```
Missing values
Wrong data types
Duplicate rows
Outliers
Inconsistent categories
Leakage columns
Unscaled features
```

Result:

```
Poor accuracy
Unstable model
Wrong predictions
Bad production performance
```

Clean data

```
Missing values handled
Duplicates removed
Outliers treated
```

```
Categories encoded
Numerical values scaled
Useful features created
Leakage avoided
```

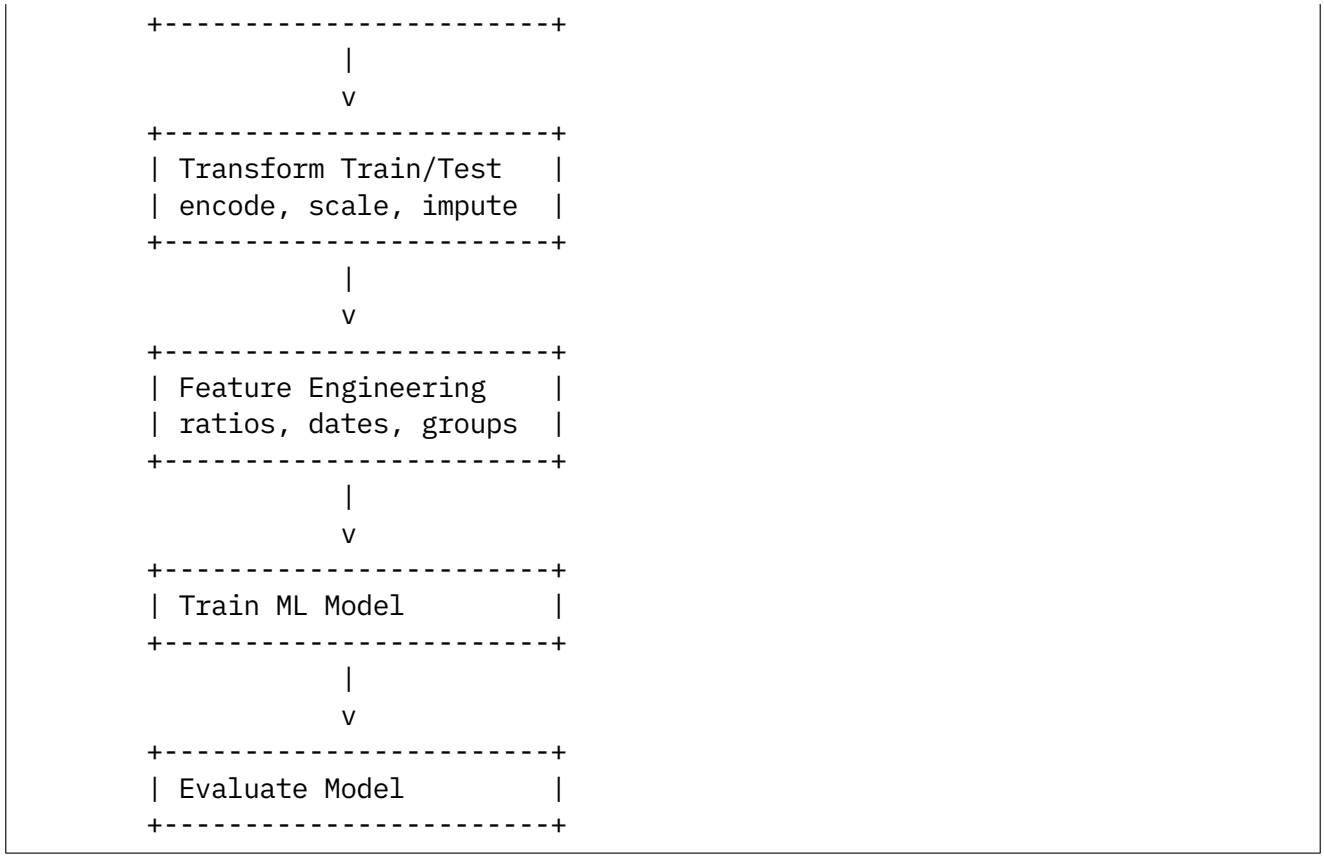
Result:

```
Better accuracy
Better generalization
More reliable model
Easier debugging
Better business trust
```

In real enterprise AI projects, data preparation often takes more time than model training.

ASCII diagram: Data preparation flow

```
                Raw Data
                   |
                   v
          +------------------------+
          | Understand Data Shape  |
          | rows, columns, dtypes  |
          +------------------------+
                      |
                      v
          +------------------------+
          | Exploratory Analysis   |
          | stats, plots, patterns |
          +------------------------+
                      |
                      v
          +------------------------+
          | Clean Data             |
          | missing, duplicates,   |
          | invalid values         |
          +------------------------+
                      |
                      v
          +------------------------+
          | Handle Outliers        |
          | remove, cap, transform |
          +------------------------+
                      |
                      v
          +------------------------+
          | Split Train/Test       |
          +------------------------+
                      |
                      v
          +------------------------+
          | Fit Preprocessing      |
          | on train data only     |
```



Pseudocode for EDA and preprocessing

```
START

Load dataset

Check shape:
    print number of rows and columns

Check columns:
    print column names

Check data types:
    identify numerical columns
    identify categorical columns
    identify date columns
    identify target column

Perform EDA:
    check summary statistics
    check missing values
    check duplicate records
    check unique values in categorical columns
    check distribution of numerical columns
    check outliers
    check target distribution

Clean data:
```

```
    fix column names
    fix wrong data types
    remove invalid rows if required
    remove duplicate records if valid

Separate features and target:
    X = input columns
    y = target column

Split data:
    X_train, X_test, y_train, y_test = train_test_split(X, y)

Fit preprocessing on training data only:
    calculate missing value replacement using X_train
    fit encoder using X_train
    fit scaler using X_train

Transform training data:
    apply imputation
    apply encoding
    apply scaling

Transform test data:
    apply same imputer
    apply same encoder
    apply same scaler

Create useful features:
    date features
    ratio features
    grouped features
    text length features

Check leakage:
    remove columns that use future information
    remove columns directly related to target
    ensure test data was not used during fitting

Train model

Evaluate model on test data

END
```

Simple Python-style example

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
```

```
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# Load data
df = pd.read_csv("loan_data.csv")

# Basic EDA
print(df.shape)
print(df.head())
print(df.info())
print(df.isnull().sum())
print(df.duplicated().sum())
print(df.describe())

# Remove duplicates
df = df.drop_duplicates()

# Separate input and target
X = df.drop("default", axis=1)
y = df["default"]

# Identify column types
numeric_features = ["age", "salary", "loan_amount"]
categorical_features = ["city", "employment_type"]

# Train/test split before fitting preprocessing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

# Numeric preprocessing
numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical preprocessing
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

# Combine preprocessing
preprocessor = ColumnTransformer(transformers=[
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])

# Full ML pipeline
model_pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor),
```

```
    ("model", LogisticRegression())
])

# Train model
model_pipeline.fit(X_train, y_train)

# Evaluate model
score = model_pipeline.score(X_test, y_test)

print("Test Accuracy:", score)
```

Important point:

```
model_pipeline.fit(X_train, y_train)
```

This ensures preprocessing learns only from training data.

Then:

```
model_pipeline.score(X_test, y_test)
```

uses the same preprocessing logic on test data safely.

Comparison table: EDA vs preprocessing vs feature engineering

Topic	Meaning	Example	Purpose
EDA	Understand data	Check missing values, distributions	Know problems and patterns
Preprocessing	Clean and convert data	Impute, encode, scale	Make data model-ready
Feature engineering	Create better inputs	loan-to-income ratio	Improve model learning
Data leakage prevention	Avoid cheating	Fit scaler only on train	Ensure realistic evaluation

---

Comparison table: Common preprocessing techniques

Problem	Technique	Example
Missing numerical values	Mean/median imputation	Fill missing salary with median
Missing categorical values	Mode/Unknown	Fill city with “Unknown”
Duplicate records	Drop duplicates	Remove repeated customers
Outliers	Cap/remove/transform	Cap extreme salary
Text categories	Encoding	Convert city to one-hot columns
Different numeric scales	Scaling	Scale salary and age
Skewed values	Log transform	Transform income
Raw dates	Date feature extraction	Extract month, day, hour

Easy end-to-end example

Suppose the business problem is:

> Predict whether a customer will default on a loan.

Raw data:

Customer ID	Age	Salary	City	Loan Amount	Default
C101	25	40000	Pune	200000	No
C102	45	90000	Mumbai	500000	Yes
C103	32	null	Delhi	300000	No
C104	500	60000	Pune	250000	No

EDA findings:

```
Salary has missing values.
Age has invalid value 500.
City is categorical.
Default is target.
Loan Amount may need scaling.
Customer ID is not useful for prediction.
```

Preprocessing:

```
Remove or fix age = 500.
Fill missing salary with median salary.
Encode city using one-hot encoding.
Scale age, salary and loan amount.
Drop customer_id.
```

Feature engineering:

```
Create loan_to_income_ratio = loan_amount / salary
```

Why useful?

A person earning ₹40,000 taking a ₹20 lakh loan may be riskier than a person earning ₹2 lakh taking the same loan.

So this feature gives better business meaning.

Data leakage example in detail

Wrong feature:

Loan Amount	Default	Recovery Status
200000	No	Not Applicable
500000	Yes	Recovery Started

If we use `Recovery Status`, the model can easily predict default.

But in real life, recovery starts after default.

So this column should not be used.

Correct thinking:

```
At prediction time, will this column be available?
```

If answer is no, remove it.

Train/test split best practice

A common beginner mistake is:

```
Clean full data
Fill missing values using full data
Scale full data
Encode full data
Then split train/test
```

This may cause leakage.

Better approach:

```
Do basic non-learning cleaning
Split train/test
Fit imputer/scaler/encoder on train only
Transform train and test separately
Train model
Evaluate model
```

Think of test data as future unseen data.

The model should not learn anything from it before evaluation.

Common mistakes

1. Skipping EDA

Beginner mistake:

```
Directly train model without checking data
```

Problem:

You may miss missing values, wrong data types, leakage columns, or duplicates.

---

2. Filling missing values without understanding reason

Example:

```
Missing salary filled with 0
```

This may be wrong because salary 0 means unemployed, not unknown.

---

3. Removing all outliers blindly

Outliers may be important.

In fraud detection, outliers may actually represent fraud.

---

4. Encoding categories incorrectly

Using label encoding for unordered categories can create fake order.

Example:

```
Pune = 1
Mumbai = 2
Delhi = 3
```

The model may think Delhi is greater than Pune, which is meaningless.

---

5. Scaling before train/test split

Wrong:

```
scaler.fit_transform(X)
```

Correct:

```
scaler.fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

---

6. Ignoring data leakage

Very high accuracy is not always good.

If accuracy is suddenly 99%, check for leakage.

---

7. Keeping ID columns

Columns like:

```
customer_id
transaction_id
policy_number
employee_id
```

usually do not help prediction.

Sometimes they cause memorization.

8. Not handling unseen categories

Example:

Training cities:

```
Pune, Mumbai, Delhi
```

Production city:

```
Hyderabad
```

If encoder cannot handle unknown values, prediction may fail.

Use:

```
OneHotEncoder(handle_unknown="ignore")
```

---

9. Not documenting preprocessing steps

In enterprise AI projects, preprocessing must be repeatable.

Bad:

```
Manual Excel cleaning
```

Good:

```
Reusable preprocessing pipeline
```

---

10. Different preprocessing in training and production

If training uses one cleaning logic and production uses another, model performance drops.

Best practice:

```
Use the same preprocessing pipeline in training, testing and deployment.
```

Enterprise AI/GenAI relevance

For IBM AI/Data Scientist roles, EDA and preprocessing are important because real enterprise data is usually messy.

In real projects, data may come from:

```
Databases
APIs
CSV files
Data lakes
Databricks tables
CRM systems
Transaction systems
```

```
Logs
Documents
```

Before using ML, GenAI, RAG, or advanced analytics, you need clean data.

Examples:

Project type	Why preprocessing matters
Traditional ML	Clean structured data improves prediction
GenAI chatbot	Clean documents improve answer quality
RAG system	Chunking, metadata and deduplication improve retrieval
Fraud detection	Outlier handling is very important
Customer churn	Feature engineering improves business signal
Databricks pipeline	Clean ETL/ELT improves downstream models
MLflow/MLOps	Reproducible preprocessing supports production deployment

For GenAI/RAG, preprocessing is not only about rows and columns.

It may include:

```
Removing duplicate documents
Cleaning text
Splitting documents into chunks
Adding metadata
Removing irrelevant content
Creating embeddings
Storing vectors in vector database
```

So the same principle applies:

> Better input data creates better AI output.

Final mental model

```
EDA tells you what is wrong or useful in the data.

Preprocessing fixes the data.

Feature engineering improves the data.

Train/test split protects honest evaluation.

Leakage prevention makes the model production-ready.
```

For interviews, remember this strong answer:

Before building any model, I first perform EDA to understand data quality, distributions, missing values, duplicates, outliers and target behavior. Then I clean the data, handle missing values, encode categorical variables, scale numerical variables where needed, and create meaningful features. I make sure that transformations like imputation, scaling and encoding are fitted only on training data to avoid data leakage. Finally, I use a reproducible pipeline so the same preprocessing is applied during training, testing and production.